

CSA15 Cloud Computing and Big Data Analytics

Capstone Cloud Technical Presentation

Individual Student Presentation Deck

Presentation Focus

SafeWalk leverages cloud computing, crowd intelligence, and real-time risk analytics to help people choose safer travel routes, enhancing personal safety and community awareness.

Presenter Details

Student Name: Konatham Mahesh

Register Number: 192311014

Class / Section / Slot: Slot B

Capstone Title: SafeWalk - Cloud Route Intelligence Using Crowd Reports and Risk Heatmaps

Selected SDG Goal(s): SDG 11 - Sustainable Cities and Communities

Cloud Platform / Tools: AWS (Lambda, API Gateway, DynamoDB, S3, Cognito)

Problem Context and Stakeholder Mapping

Our capstone begins with a real problem to solve

Problem Statement

Standard map apps optimize for distance and time, not safety - so pedestrians, especially at night, unknowingly walk through poorly lit or high-incident areas with no warning.

Target Users

Students, women commuting alone, night-shift and gig workers, and city safety planners; they gain a risk-aware route choice instead of a purely shortest-path one.

SDG Fit

SDG 11: Sustainable Cities and Communities - safer, more inclusive public spaces and mobility for all residents.

Product Boundary

MVP covers one city zone: crowd-sourced incident reporting plus a risk-scored route suggestion; law-enforcement integration is out of scope for v1.

Cloud Adoption Rationale and Concept Map

The problem needs cloud support to become a usable product

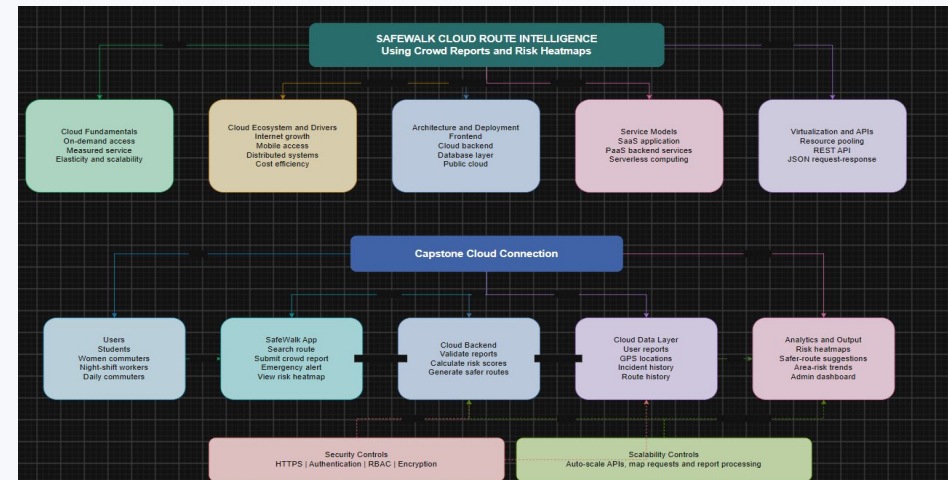
Cloud Definition

For SafeWalk, cloud means on-demand compute for risk scoring, elastic storage for a growing pool of crowd reports, and reliable delivery of routes to any user, anywhere, without managing servers.

Cloud Characteristics

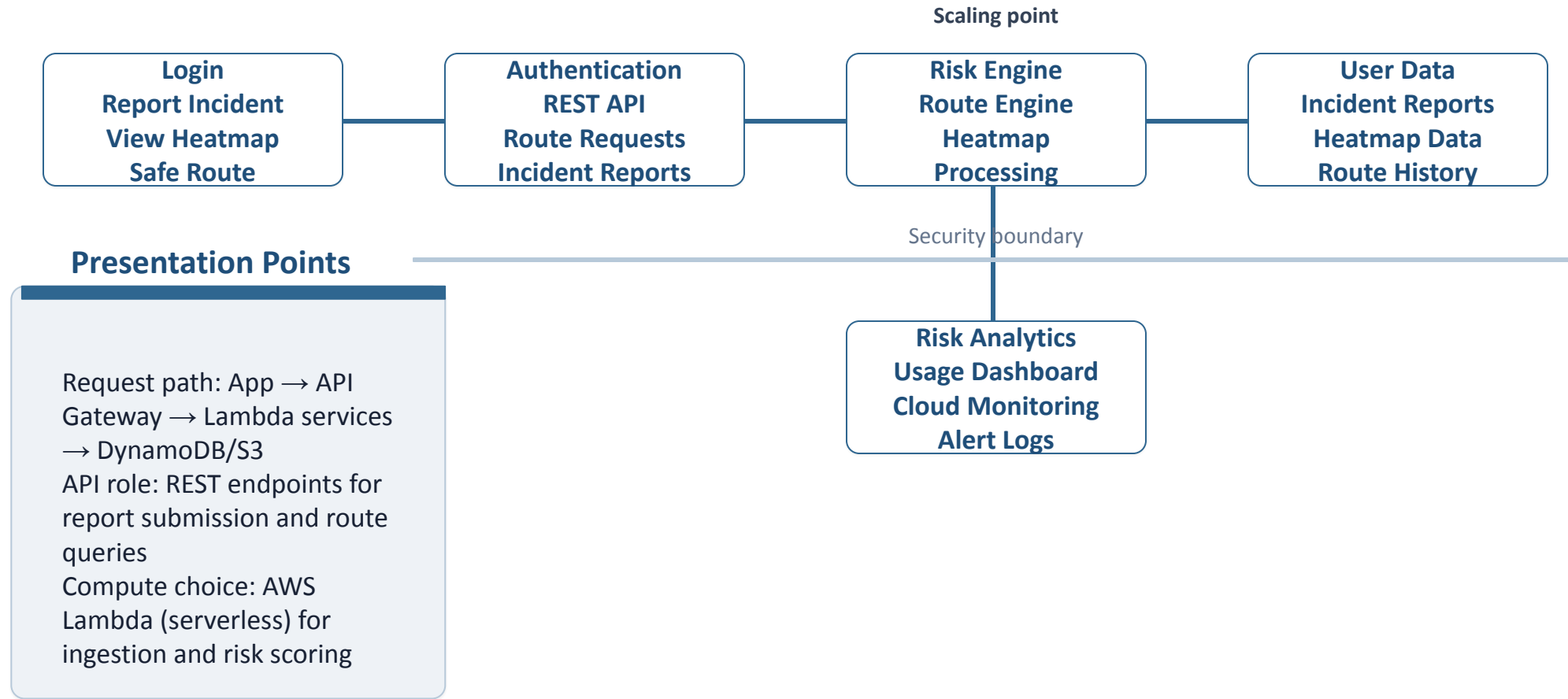
Elasticity absorbs report spikes during events; scalability lets SafeWalk grow city by city; measured service keeps low-traffic zones cheap; resource pooling shares managed DB/compute across all users.

Cloud Concept Map



Capstone Cloud Architecture

Our cloud architecture connects the user action to services



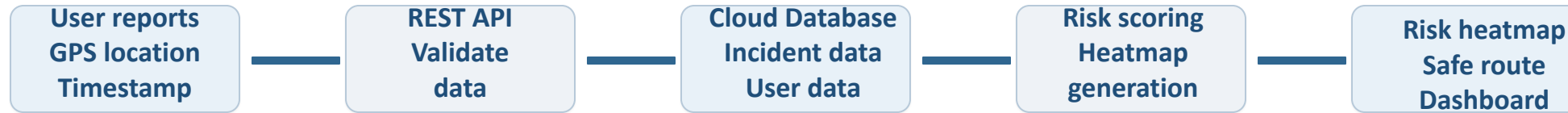
Cloud Service Model Responsibility Matrix

Each service model defines what we control and what cloud manages

Service area	Model	Provider manages	Student team manages	Why this fits
Frontend hosting	SaaS-style (Amplify Hosting)	Hosting infra, CDN, SSL certs	React app code, UI/UX logic	Fast static deploys let us focus on map UX, not servers
Backend/API	PaaS (API Gateway + Lambda)	Servers, patching, auto-scaling	Business logic, API contracts	Serverless fits unpredictable report traffic
Database	PaaS (DynamoDB)	Replication, scaling, backups	Table/schema design, queries	Fast key-value access for real-time reports
Storage	PaaS (Amazon S3)	Durability, availability	Bucket structure, lifecycle rules	Cheap, durable storage for heatmap tiles
Monitoring / logs	SaaS (CloudWatch)	Log infra, dashboards engine	Custom alarms, metrics	Out-of-box observability, no setup needed
Security / IAM	PaaS (IAM + Cognito)	Auth infra, token issuance	Roles, policies, user pools	Secure auth without building it from scratch

Data Flow and Analytics Pipeline

The product data moves from user activity to useful insight



Data Requirements

Crowd incident reports (type, GPS location, timestamp, optional photo) and app-side GPS pings during navigation.

Analytics Requirement

Live risk heatmap per city zone, a route safety score, and a trend alert when incidents spike in an area.

Proof Required

Input: report JSON with lat/long and type.
Output: risk score per grid cell. Insight: poor-lighting reports cluster near one street between 8-10pm.

Python / AI Module Integration

Python or AI adds intelligence where the product needs it

Python Role

A Python risk-scoring model weighs report recency, severity, time-of-day, and density to score each geohash grid cell.

Input and Output

Input: new incident reports (lat/long, type, time). Output: an updated risk score per grid cell and a re-ranked list of route options.

Execution Point

AWS Lambda triggered by DynamoDB Streams on each new report, plus a scheduled EventBridge job that refreshes the full heatmap every few minutes.

Product Value

Turns scattered reports into an actionable safety layer over routes, so users can trade a little distance for a lot more safety.

Security, Scalability, Privacy and Cost Design

Security, scale, privacy and cost shape the final design

Security

Cognito-based sign-in, input validation on report submissions, signed S3 URLs for photos, and CloudWatch audit logs of report edits.

Privacy

Reports are anonymous by default, coordinates are fuzzed by ~50m, and photo evidence is opt-in with explicit consent.

Scalability

Serverless compute and on-demand DynamoDB auto-scale with report volume, so new city zones can be added without redesign.

Cost Awareness

AWS Free Tier covers Lambda, DynamoDB, and S3 during development; pay-per-request pricing keeps low-traffic zones nearly free.

Implementation Stack and Deployment Evidence

Our implementation stack shows how the design will be built

Mobile / Web Layer

React web app (or Flutter mobile) with a map view for route search, risk overlay, and one-tap incident reporting.

Cloud Services

API Gateway, Lambda, DynamoDB, S3, Cognito, EventBridge, and CloudWatch on AWS.

Evidence

GitHub repository, a Postman API test screenshot, the CloudWatch dashboard, and the deployed API endpoint URL.

Version Control

Repo split into /frontend, /backend, and /infra with a clear README, feature branches, and descriptive commits.

Demo Storyboard and User Journey

A complete user journey proves the product flow

Scene 1

Opens app, allows location -> map
centers on current position -> Client GPS
+ Cognito auth

Scene 2

Requests route to destination ->
candidate routes computed -> API
Gateway + Route Lambda

Scene 3

Views color-coded risk overlay on routes
-> heatmap renders on map -> Risk
Engine Lambda + DynamoDB

Scene 4

Picks the safer, slightly longer route ->
choice logged for analytics -> Backend
Services + CloudWatch

Scene 5

Spots a poorly lit alley, submits a report ->
report saved instantly -> Report Lambda +
DynamoDB Streams

Scene 6

Next user queries the same area -> sees
the updated heatmap -> S3 heatmap
cache refresh

Evaluation Readiness Checklist

Our evidence confirms that the presentation is ready

☐

Cloud ecosystem and service model explained

☐

Elasticity and scalability decisions are visible

☐

REST/SOAP/API role connected to capstone

☐

GitHub and cloud evidence are inspectable

☐

Architecture has compute, data, storage, security, monitoring

☐

References and integrity statement are included

Evaluator Focus

CO1 is verified through cloud design reasoning, service choices, evidence, and individual explanation.

Conclusion, Limitations and Future Scope

We close with learning, limitations and next steps

Learning Outcome

Designing an event-driven serverless architecture, applying elasticity to a real safety problem, and splitting responsibility between provider-managed and self-managed services.

Limitations

Reports are currently unverified with no moderation, and the heatmap is less accurate in low-traffic zones with fewer reports.

Future Improvement

Add a lightweight ML classifier for report moderation, integrate historical incident data, and add push alerts for rising-risk zones.

Integrity Statement

Built with AWS documentation and course material; presentation content and structure drafted with Claude AI assistance and reviewed by the student.